

universidad de León Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Introducción y motivación

Francisco Javier Rodríguez Lera ▪ Camino Fernández Llamas



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías

universidad de León

Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Distinguir** código funcional de código correcto, robusto, seguro, portable y mantenible.
- **Formular** precondiciones, postcondiciones e invariantes y aplicarlos a un caso de procesamiento de archivos.
- **Identificar** por CWE familias clásicas de vulnerabilidad en código humano o generado por IA.
- **Explicar** por qué superar una batería finita de pruebas no demuestra seguridad.
- **Diseñar** un protocolo reproducible para comparar sistemas de IA.
- **Diferenciar** un sistema de IA regulado de software producido con IA.
- **Distinguir** *sandbox* regulatorio, aislamiento técnico y *pipeline* de validación.
- **Modelar** amenazas de las acciones de un agente.
- **Diseñar** un *pipeline* de validación con puertas y trazabilidad.
- **Aplicar** la metodología docente AI-SDLC y reconocer sus límites.

La promesa y su medida

La promesa comercial es atractiva y fácil de resumir: *escribir software cuesta menos*. Pero mezcla magnitudes distintas.

- **Producir** líneas de código $\neq$ **entregar** software correcto, robusto, seguro, portable y mantenible.
- La herramienta *no* decide qué es aceptable.
- La responsabilidad profesional recae en el equipo humano que firma el artefacto.

DISCUSIÓN ¿Qué se abarata realmente?

- ¿Qué coste reduce la IA generativa: el de *escribir* o el de *entregar*?
- Si la IA asume parte de la escritura, ¿qué tareas humanas se vuelven *más* importantes?

Hipótesis rectora

Hipótesis. La IA generativa **reduce ciertos costes de implementación** -escritura inicial, andamiaje, traducción, generación de pruebas- **sin eliminar** los problemas de correctitud, seguridad, mantenimiento, portabilidad y responsabilidad.

- No se asume que la IA mejore ni empeore la seguridad *per se*.
- Desplaza el esfuerzo humano hacia la **especificación, la revisión y la validación**.
- Añade **superficie de ataque nueva** en la cadena de herramientas.

La hipótesis es asimétrica

- **Evidencia parcial** (contextualizada): la productividad depende del contexto. El estudio de METR (2025) halló una *ralentización* en su entorno experimental, no una ganancia universal [1].
- **Niega** que la ganancia de productividad se traduzca automáticamente en garantías de calidad o de seguridad.
- El capítulo pone a prueba esa asimetría en cada etapa del desarrollo.

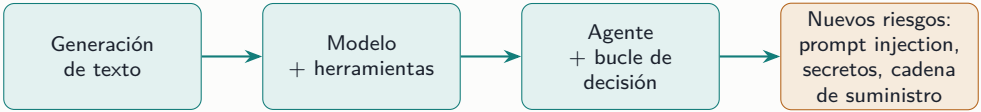
Superar los casos de prueba elegidos no demuestra ausencia de vulnerabilidades [2].

Por qué importa en Diseño y Programación Seguras

Si la IA asume parte de la escritura, el trabajo humano se concentra en tareas que antes se daban por supuestas. Dos consecuencias estructuran el capítulo:

1. **El esfuerzo se mueve hacia la especificación y la validación.** Precondiciones, postcondiciones e invariantes pasan a ser el instrumento principal de control de calidad.
2. **Aparece superficie de ataque nueva.** Con herramientas concedidas (comandos, red, escritura, paquetes), el asistente se convierte en un *agente*: inyección de *prompt*, exfiltración de secretos, dependencias maliciosas.

Del asistente al agente



Un asistente de código no es solo un generador de texto: al concederle herramientas se convierte en un agente con capacidad de actuar sobre el entorno.

Advertencia de seguridad

No ejecute código no confiable -generado por una IA o por terceros- fuera de un entorno aislado y autorizado. Los ejemplos vulnerables del libro son material didáctico: se estudian y, si se ejecutan, se hace únicamente en un *sandbox* sin acceso a datos reales, red ni credenciales.

Caso transversal: de CSV a PDF/Excel

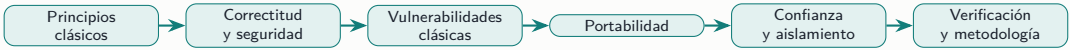
Para dar continuidad se reutiliza un mismo escenario: convertir un fichero **CSV** en **PDF** y **Excel (XLSX)** con validación previa.

- **Realista sin ser trivial**: cabe en una sesión.
- Obliga a tratar a la vez **entrada no confiable**, **salida a disco** y **dependencias externas**.
- Permite comparar, sobre el mismo enunciado, código vulnerable, corregido y generado por IA.

La descripción completa se desarrolla en el Anexo A (caso de estudio transversal); los capítulos posteriores lo utilizan sin repetirla.

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

El caso como hilo conductor



Objetivos pedagógicos: dar un escenario concreto y compartido a los conceptos abstractos de especificación, verificación y riesgo; y servir de hilo conductor entre capítulos.

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

Invariante terminológico

La discusión se degrada cuando se confunden niveles de abstracción. Distinción estricta, usada en todos los capítulos:

Concepto	Qué es	No confundir con
Modelo	Pesos y arquitectura (una familia de LLM)	El producto o el asistente
Versión	Identificador concreto del modelo	«El modelo» genérico
Configuración	Temperatura, límites, *system prompt*	El modelo en sí
Agente	Modelo + herramientas + bucle	Generación aislada
Herramientas	Comandos, red, escritura, paquetes	El modelo
Entorno	SO, arquitectura, contenedor, *sandbox*	El modelo o el agente

Distinción *operativa*, no erudita: sin ella no se puede reproducir un experimento ni atribuir responsabilidades.

Estructura del libro

Capítulos

- 1. Introducción y motivación
- 2. Principios clásicos y diseño seguro
- 3. Correctitud y seguridad
- 4. Vulnerabilidades clásicas
- 5. Portabilidad y plataformas
- 6. Evaluación reproducible de sistemas de IA
- 7. Reglamento de IA y espacios controlados
- 8. Confianza no calibrada
- 9. Aislamiento técnico y agentes
- 10. Pipeline de validación automática
- 11. Verificación de código generado
- 12. Metodología docente AI-SDLC
- 13. Conclusiones
- Bibliografía y fuentes

Anexos

- A. Caso de estudio transversal
- B. Prácticas de laboratorio
- C. Plantillas de registro

Cómo leer: etiquetado epistémico

Toda afirmación relevante se etiqueta, cuando procede:

Evidencia verificable en fuentes.

Hipótesis conjetura razonada aún no demostrada.

Propuesta docente decisión pedagógica, no resultado científico.

Esta disciplina es *parte del contenido*: en un campo donde abundan las afirmaciones sin respaldo, distinguir lo demostrado de lo conjeturado es una competencia profesional.

Cautela invariante

Superar una batería finita de pruebas **no demuestra** que un programa sea seguro. Un ejemplo «corregido» es correcto de forma *contextual*: lo es respecto de las amenazas y precondiciones declaradas, y *nunca* «universalmente seguro».

- Esta cautela se repite a lo largo de todo el libro.
- Es el invariante metodológico que conecta todos los capítulos.

Síntesis del capítulo

- La IA generativa **abarata escribir**, no **entregar**.
- El esfuerzo humano se desplaza a **especificar, revisar y validar**.
- Conceder herramientas convierte al asistente en **agente**: nuevos riesgos.
- Un **invariante terminológico** permite reproducir y atribuir.
- El **caso transversal CSV–PDF–XLSX** da continuidad a todo el libro.
- Toda afirmación se etiqueta como **Evidencia**, **Hipótesis** o **Propuesta docente**.

Actividad

1. Clasifique tres afirmaciones habituales sobre IA y programación como *Evidencia*, *Hipótesis* o *Propuesta docente*, y justifique.
2. Explique, con un ejemplo, la diferencia entre *producir* y *entregar* software.
3. Enumere tres riesgos que aparecen al conceder herramientas a un asistente y no existen en un editor de texto convencional.

¿Es la productividad una garantía de seguridad?

- ¿Qué evidencia justificaría afirmar que un asistente «mejora» la seguridad?
- ¿Cómo distinguir una ganancia de velocidad de una ganancia de calidad?

Referencias I



Joel Becker, Nate Rush, Beth Barnes, and David Rein.

Measuring the impact of early-2025 ai on experienced open-source developer productivity.

Informe experimental, Model Evaluation & Threat Research (METR), July 2025.

Estudio contextualizado (RCT con desarrolladores experimentados, 2025); halló una ralentización en su contexto; no es un efecto universal.



Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kelly Pei, Ofir Press, and Karthik Narasimhan.

Swe-bench: Can language models resolve real-world github issues?

In *International Conference on Learning Representations (ICLR)*, 2024.

Publicación científica; superar sus pruebas no demuestra ausencia de vulnerabilidades.